

PharmSignals — Complete Repository Overview

Drug Safety Signal Detection & Regulatory Submission Readiness

Team: Demon Slayer | Track: AI | Problem Statement: P2 | IBM Bobathon 2026

What the Project Is

PharmSignals is a full-stack clinical intelligence and regulatory compliance platform with two core workflows:

1. Post-marketing Pharmacovigilance — detects drug safety signals from real openFDA FAERS adverse event data using statistical disproportionality analysis.
2. Pre-marketing Regulatory Dossier Compliance — validates ICH M4 CTD dossier readiness against authoritative guideline requirements across Modules 1–5.

Both workflows are accessible through a unified white-theme enterprise UI with an embedded IBM Bob AI Copilot.

Backend — Python FastAPI (src/backend/)

Module M1 — FAERS Data Ingest

Status: Fully Implemented

Loads real openFDA FAERS adverse event data from merged_clean.csv. Normalises drug names to uppercase (MedDRA-style). Validates records — drops empty rows, clamps negative counts. Builds drug-event contingency tables from per-drug signal CSVs (VIOXX, BAYCOL, AVANDIA). The full M1 pipeline is callable via run_m1_pipeline().

Module M2 — PRR Signal Engine

Status: Fully Implemented

Computes Evans Proportional Reporting Ratio (PRR) using $a/(a+b)$ divided by $c/(c+d)$. Computes Pearson Chi-Square with SciPy's chi2_contingency. Computes log-normal 95% Confidence Intervals. Classifies signals as SIGNAL / WEAK_SIGNAL / NOISE using FDA thresholds (PRR ≥ 2.0 , Chi-Square ≥ 4.0 , cases ≥ 3). Supports bulk run_prr_analysis() over full contingency tables, results ranked by PRR descending.

Module M3 — Digital Twin Backtest

Status: Fully Implemented

Loads pre-computed monthly PRR trajectory CSVs for VIOXX, BAYCOL, and AVANDIA. Reads backtest_summary.csv for detection dates, lead times, and verdicts. Key results:

- VIOXX: +242 days early detection before 2004-09-30 market withdrawal.
- AVANDIA: +1,205 days early detection before 2007-05-21 FDA Boxed Warning.
- BAYCOL: Transparently reports DATA_UNAVAILABLE_PRE_WITHDRAWAL (openFDA starts 2004; Baycol withdrawn 2001).

Includes `get_emerging_signals()` for trend-based early warning and drug alias resolution (ROFECOXIB → VIOXX, etc.). Trajectory plots are pre-generated as PNGs.

Module M4 — ICH M4 RAG CTD Readiness Checker

Status: Fully Implemented (most complex module)

The M4 module is a multi-component RAG (Retrieval-Augmented Generation) pipeline for validating CTD dossier readiness. Key components:

- `schema.py` — Pydantic models: `DossierOutlineInput`, `GapItem`, `GapReportOutput`, `ICHSectionRequirement` with enums for `GapStatus`, `CriticalityLevel`, `MatchMethod`.
- `knowledge/ich_m4_sections.json` — Authoritative ICH M4 section requirements for Modules 1–5.
- `knowledge/loader.py` — In-memory knowledge base with `get_by_module()`, `get_all()`.
- `retriever.py` — `ICHRetriever` with hybrid exact + semantic section matching.
- `gap_checker.py` — Evaluates candidate dossier sections against ICH requirements, produces `GapItem` list.
- `completeness.py` — Weighted completeness score calculator (overall %, per-module).
- `dossier_parser.py` — Parses structured JSON, plain text, or raw strings into `DossierOutlineInput`.
- `pdf_extractor.py` — `CTDPDFExtractor` using `pdfplumber`/`pypdfium2` to extract sections from uploaded PDFs.
- `gemini_reasoner.py` — `GeminiGroundedReasoner`; optional Gemini 2.5 Flash reasoning with deterministic fallback.
- `report_generator.py` — Assembles final `GapReportOutput` with priority gaps, recommended actions, limitations.
- `presets.py` — 3 pre-packaged dossiers: Vioxx NDA 21-042, BOB-701 Oncology, Phase 1 IND (EXP-101).
- `pipeline.py` — `CTDRagCheckerPipeline`: 5-step orchestrator (parse → evaluate → score → reason → report).

REST API — 17 Endpoints Across 3 Routers

Status: Fully Implemented

GET	<code>/api/v1/health</code>	Health check for all subsystems
GET	<code>/api/v1/signals/</code>	List all PRR results (drug/status filters)
GET	<code>/api/v1/signals/summary</code>	Total pairs, confirmed signal count, top 10 signals
GET	<code>/api/v1/signals/drugs</code>	List available drugs in dataset
POST	<code>/api/v1/signals/calculate</code>	Custom 2x2 PRR calculator (a,b,c,d) or margin inputs
GET	<code>/api/v1/signals/trajectory/{drug}/{event}</code>	Monthly PRR time-series for a drug-event pair
GET	<code>/api/v1/signals/vioxx-backtest</code>	Vioxx-specific backtest endpoint
GET	<code>/api/v1/signals/backtest/{drug}</code>	Generic backtest for VIOXX / BAYCOL / AVANDIA
POST	<code>/api/v1/m4/check</code>	Evaluate structured JSON dossier outline
POST	<code>/api/v1/m4/quick-check-text</code>	Evaluate raw plain-text dossier outline
POST	<code>/api/v1/m4/check-pdf</code>	Upload and evaluate a PDF dossier
GET	<code>/api/v1/m4/presets</code>	List all pre-packaged dossiers
GET	<code>/api/v1/m4/presets/{key}</code>	Get specific preset

GET	/api/v1/m4/requirements	Full ICH M4 knowledge base
GET	/api/v1/m4/module/{id}	ICH requirements for a specific module (1–5)
POST	/api/v1/copilot/query	Ask IBM Bob (Gemini or deterministic fallback)
GET	/api/v1/copilot/suggestions	Curated starter prompts

IBM Bob AI Copilot — Backend

Status: Fully Implemented with Dual-Engine Design

Builds domain-specific context from live backtest data, PRR stats, and ICH knowledge base. Primary engine calls Gemini 2.5 Flash API via httpx when GEMINI_API_KEY is present. Fallback is a deterministic expert rule-based engine covering Vioxx, Avandia, Baycol, PRR formula, ICH Modules 3/4/5, gap scoring — zero hallucinations guaranteed offline.

Frontend — Next.js 14 + TypeScript + Tailwind + Recharts (src/frontend/)

Status: Fully Implemented

The entire UI lives in one large page component (page.tsx, ~1,864 lines) with 6 distinct views:

1. Dashboard — Metric cards (total signals, confirmed signals), Adverse Event Bubble Chart (PRR vs case count, colored by signal status using Recharts ScatterChart + ZAxis).
2. Signal Detection — Full PRR signals table with filter/sort, Interactive 2x2 Contingency Table Calculator with 4 preset buttons (Vioxx MI, Baycol Rhabdo, Avandia Heart Failure, Ibuprofen non-signal), custom drug/event input, real-time result display.
3. Historical Analysis — Drug selector (VIOXX / BAYCOL / AVANDIA), PRR time-series LineChart with signal threshold reference line, lead time badge, clinical summary, verdict display.
4. Submission Readiness — Preset dossier selector + raw text textarea + PDF file upload, module progress bars (Modules 1–5), overall readiness percentage, Priority Regulatory Gap Matrix with CRITICAL/MAJOR/STANDARD filter.
5. Reports — Download JSON export of FAERS signal data (M1/M2), navigation links to Backtest and Readiness workspaces.
6. About — Platform architecture overview.

Reusable Shared Components:

- Sidebar.tsx — Collapsible left navigation.
- Header.tsx — Top bar with backend status indicator and Bob Copilot toggle.
- BobCopilotDrawer.tsx — Slide-in right drawer with chat history, markdown rendering, follow-up chips, quick suggestion chips.
- MetricCard.tsx, StatusBadge.tsx, PageHeader.tsx, LoadingState.tsx, ErrorState.tsx, EmptyState.tsx — Shared UI primitives.

Test Suite — src/backend/tests/ (Pytest)

Status: 114 Automated Tests Across 16 Test Files

- test_health.py — Backend health endpoints.
- test_m1_faers.py — FAERS ingest, normalization, contingency table building.
- test_m2_prr.py — PRR calculation, chi-square, signal classification.
- test_m3_trajectory.py — Trajectory loading, backtest verdicts.
- test_m4_*.py (12 files) — Schema, knowledge loader, retriever, gap checker, completeness, dossier parser, PDF extractor, pipeline, report generator, Gemini reasoner, API endpoints, deep QA.
- test_copilot_and_custom_calc.py — Copilot queries and custom PRR endpoint.

Data Artifacts — src/backend/app/data/

- merged_clean.csv — Real openFDA FAERS master adverse event dataset.
- m2_signals/VIOXX_signals.csv, BAYCOL_signals.csv, AVANDIA_signals.csv — Pre-computed 2x2 contingency cells per drug.
- m2_signals/all_signals_ranked.csv — All signals ranked by PRR.
- m3_digital_twin/VIOXX_trajectory.csv, BAYCOL_trajectory.csv, AVANDIA_trajectory.csv — Monthly PRR walk-forward time series.
- m3_digital_twin/backtest_summary.csv — Detection dates, lead times, and verdicts per drug.
- m3_digital_twin/trajectory_plots/ — Pre-rendered PNG trajectory charts.
- m4_rag/knowledge/ich_m4_sections.json — Full ICH M4 Modules 1–5 section requirements.

Technology Stack

- Frontend: Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, Recharts
- Backend API: Python 3.10+, FastAPI, Uvicorn, Pydantic v2
- Statistical Engine: NumPy, SciPy, Pandas
- RAG & Knowledge Base: In-Memory Retrieval Index (ICH M4 guideline retrieval)
- AI Copilot: IBM Bob / Google Gemini 2.5 Flash / watsonx.ai (configured)
- PDF Processing: pdfplumber, pypdfium2
- DevOps: Concurrently, Pytest (114 tests), Node.js
- CI/CD: GitHub Actions (.github/workflows/validate.yml)

Known Gaps / Not Yet Done

1. Adverse Event Clustering — Explicitly noted as disabled in README; labeled 'in development' rather than showing fabricated groupings.
2. Demo video link — demo/demo-video-link.txt is a placeholder; no actual video recorded yet.
3. Live deployment URL — demo/live-demo-url.txt is a placeholder; the app is not deployed to any cloud host.

4. Unused dependencies — psychopg2-binary and SQLAlchemy are listed in requirements.txt but no database models or ORM usage exists in the codebase; all data processing is in-memory.
5. watsonx.ai integration — WATSONX_API_KEY and WATSONX_PROJECT_ID env vars are defined and documented, but no watsonx API call exists in any backend file; only Gemini 2.5 Flash is wired up as the live LLM path.